PRACTICAL- 4

AIM

Write a python code for Optimal Problem-Solving using AO* Algorithm.

THEORY

The AO* (And-Or Star) algorithm is an informed search algorithm used in Artificial Intelligence for solving problems that can be represented using AND-OR graphs. Unlike traditional search algorithms such as BFS or DFS that work on simple graphs, AO* is designed to handle problems where a solution may require satisfying multiple subproblems simultaneously (AND nodes) or choosing one among several alternatives (OR nodes).

In an AND-OR graph:

- OR nodes represent alternative choices where solving any one child node is sufficient.
- AND nodes represent situations where all child nodes must be solved to reach the solution.

The AO* algorithm uses heuristic values to estimate the cost from a node to the goal state. These heuristic values guide the search toward the most promising path, helping reduce unnecessary exploration. The algorithm evaluates the cost of nodes and repeatedly expands the most promising node based on the minimum estimated cost.

The AO* algorithm works by:

1. Starting from the initial node.
2. Expanding nodes by selecting the child nodes with the lowest heuristic cost.
3. Updating the cost of parent nodes as better solutions are found.
4. Continuing the process until the goal node is reached and the optimal solution path is identified.

The main advantage of AO* is that it efficiently finds the optimal solution in AND-OR graphs while minimizing unnecessary search operations. Because of its ability to handle complex problem structures, it is commonly used in problem reduction systems, planning problems, and decision-making applications in Artificial Intelligence.

SOFTWARE USED

Jupyter Notebook

CODE

class AOStar:

```
def __init__(self, graph, heuristic):
    self.graph = graph
    self.h = heuristic

def find_best_child(self, node):

    min_cost = float('inf')
    best_child = None

    print("\nEvaluating children of node:", node)

    for child in self.graph.get(node, []):

        cost = self.h[child] + 1
        print("Child:", child, " Heuristic:", self.h[child], " Cost:", cost)

        if cost < min_cost:
            min_cost = cost
            best_child = child

    print("Best Child Selected:", best_child)

    return best_child

def ao_star(self, start):

    current = start
    print("\nStart Node:", current)
```

```
        while current in self.graph and len(self.graph[current]) > 0:

            next_node = self.find_best_child(current)

            print("Move to Node:", next_node)

            current = next_node

        print("\nGoal Node Reached:", current)



graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

heuristic = {
    'A': 3,
    'B': 2,
    'C': 1,
    'D': 0,
    'E': 0,
    'F': 0
}
```

```
start_node = input("Enter Start Node: ")

obj = AOStar(graph, heuristic)

obj.ao_star(start_node)
```

OUTPUT

```
Enter Start Node:  A

Start Node: A

Evaluating children of node: A
Child: B  Heuristic: 2  Cost: 3
Child: C  Heuristic: 1  Cost: 2
Best Child Selected: C
Move to Node: C

Evaluating children of node: C
Child: F  Heuristic: 0  Cost: 1
Best Child Selected: F
Move to Node: F

Goal Node Reached: F
```